

OPNaaS Product and Developer Manual

Final Sprint Recap

Below is a recap of the responsibilities and features covered in our previous sprint.

Responsibilities

- Feature 1: Resource Dashboard
 - Dan McCarthy
 - Kestutis Biskis
- Feature 2: Virtualization Integration
 - Evan Parker
 - Matt Gee
 - Alex Houle

Backlog Review

Resource Dashboard

Item	Status	Discussion
View existing resources on dashboard	DONE	n/a
Add new resources to dashboard	DONE	n/a
Manage resources on dashboard	DONE	n/a

Github Issues:

- OPNaaS #3 (Create landing page)
- OPNaaS #4 (Create login page)
- OPNaaS #5 (Create Public Dashboard)
- OPNaaS #6 (Create Admin Dashboard)
- OPNaaS #7 (Setup host API on backend)
- OPNaaS #13 (Setup tailwind on frontend)
- OPNaaS #28 (Add new website logo)
- OPNaaS #62 (Improve admin dashboard)

Virtualization Integration

Item	Status	Discussion
Create virtualized containers	DONE	Functional on backend, needs to be added to frontend

Add virtualization to frontend	IN PROGRESS	n/a
--------------------------------	-------------	-----

Github Issues:

- OPNLaaS #20 (Proxmox virtualization integration)
- OPNLaaS #21 (Handle ISO upload)
- OPNLaaS #48 (Add support for PXE boot)
- OPNLaaS #65 (Add virtualization to frontend)

Developer Documentation

Our project can be accessed at:

- <https://laas.cyber.lab/>
- **NOTE: This is deployed locally in the Cybersecurity Lab. To access to the site you will need to be on the Cybersecurity VPN.**

Our Github repo and related issues can be found below:

- <https://github.com/OpnLaaS/opnlaas>
- <https://github.com/OpnLaaS/opnlaas/issues>

Repository Overview

```
opnlaas
├── app
├── auth
├── config
├── db
├── docs
├── host
├── node_modules
├── public
├── scripting
├── ssh
├── tests
└── vm
```

/App

Contains the backend code that immediately talks to the frontend and receives web requests, contains:

- Defined routes on the backend
- Primary functions called by the routes

- Api routes and views are in separate files
- Any needed middleware for the routes

/Auth

Responsible for managing the authentication pipeline between the remote (network) ldap and the browser.

- Contains ability to "inject" a user to become authenticated for testing, but this is not used for production code

/Config

Loads, types, and verifies initial configuration values from our config.toml file.

/DB

The database management for our codebase, contains defined database schemas and functions to initialize the local database when the backend starts.

/Docs

You're already here!

/Host

Software to interface with, and manage hosts; whether they are virtual or physical.

/Host/Iso

Files and scripts for parsing and extracting relevant content from uploaded iso files, these files are then stored to be used when a host is attempting to pxe boot.

/Host/PXE

Contains the code to automate the provisioning process of pxe booting a server with an automated install of an operating system. Contains files to run necessary services other than http and templating functions and files in order to create auto-configuration files such as kickstart, autoinstall, and grub commands.

/Public

Contains content served to the browser, including CSS, Javascript, images, and html templates. The content is primarily served from the app directory.

/Scripting

Script utilities that are not run automatically as they server "one time" functionality for developers and maintainers.

/SSH

Files for managing ssh connections and keys automatically to allow the backend to create ssh sessions and use them as needed.

/Tests

Contains all of the backend tests for the project, these are running automatically by the github actions runner configured in .github.

/VM

Files for management, creation, and deletion of virtual resources through the Proxmox api to the local proxmox session in the Cybersecurity lab.

Running the Project Locally

To run the project locally you need to do a few things:

- Clone the git repo
- Install Go and Node.js:
 - Go version 1.24.0 or higher
 - npm version 10.0.0 or higher
- Setup a config.toml file

If all of this is setup you should be able to run the following command to setup the frontend:

```
npm run build
```

After that all that is left is running the backend using this command:

```
go run main.go
```

Assuming everything is setup correctly you should see something like this:

```
danmc@laptop dev/opnlaas (86-documentation *) » go run main.go
[DB] [+] Database initialized!
```

```
Fiber v2.52.10
http://127.0.0.1:8080
(bound on host 0.0.0.0 and port 8080)

Handlers ..... 93   Processes ..... 1
Prefork ..... Disabled PID ..... 595810
```

Setting up config.toml

If you run `go run main.go` without having a `config.toml` file setup you should run into the following error:

```
danmc@laptop dev/opnlaas (86-documentation *) » go run main.go
[MAIN] [!] Failed to initialize environment: no config file found, created a default config at /home/danmc/dev/opnlaas/config.toml. Please fill in the required values and try again
panic: no config file found, created a default config at /home/danmc/dev/opnlaas/config.toml. Please fill in the required values and try again
```

When this happens a blank `config.toml` will be created that looks like this:

```
[web_server]
  address = ":8080"
  tls_dir = ""
  reload_templates_on_each_render = false
  redirect_server_addresses = []

[ldap]
  address = ""
  domain_sld = ""
  domain_tld = ""
  accounts_cn = "accounts"
  users_cn = "users"
  groups_cn = "groups"
  admin_groups = ["admins"]
  user_groups = ["ipausers"]

[management]
  username = ""
  password = ""
  [management.testing]
    [management.testing.basic]
      enabled = false
      ips = []
    [management.testing.long]
      enabled = false
```

```
        ip = ""

[database]
    file = "laas.db"

[proxmox]
    enabled = false
    hostname = ""
    port = ""
    token_id = ""
    secret = "laas-api-token-secret"
    [proxmox.testing]
        enabled = false
        subnet_cidr = "10.255.255.0/24"
        storage = "local-lvm"
        ubuntu_template = "local:vztmpl/ubuntu-22.04-standard_22.04-
        gateway = "10.0.0.1"
        dns = "10.0.0.2"
        search_domain = "local"

[isos]
    search_dir = "./isos_search_dir"
    storage_dir = "./isos"
    testing = false

[pxe]
    enabled = false
    [pxe.dhcp_server]
        address = ":67"
        interface = "eth0"
        server_public_address = ""
        proxy_mode = false
        ip_range_start = ""
        ip_range_end = ""
        lease_seconds = 7200
        subnet_mask = ""
        router = ""
        dns_servers = []
    [pxe.tftp_server]
        address = ":69"
        directory = "/var/lib/tftpboot"
    [pxe.http_server]
        address = ":8069"
        directory = "/var/www/tftpboot"
```

```

    public_url = ""

[preconfigure]
    locale = "en_US"
    timezone = "UTC"
    keyboard_layout = "us"
    keyboard_variant = ""
    packages = ["openssh-server"]
    mirror = ""
    root_password = ""
    disable_root = true
    global_kernel_params = []
    global_initrd_params = []
[preconfigure.managed_user]
    username = "laas-admin"
    password = "laas-admin"
    allow_sudo = true
    ssh_authorized_keys = []
[preconfigure.given_user]
    username = "laas"
    password = "laas"
    allow_sudo = true
    ssh_authorized_keys = []
[preconfigure.scripting_file_paths]
    global_pre_script_file = ""
    global_post_script_file = ""

```

Most of this config can be left the same. The **only things you will need to change** are the ldap address/domain, and the management user/password as shown below:

```

[ldap]
    address = "ldaps://domain.cyber.lab:636"
    domain_sld = "cyber"
    domain_tld = "lab"

[management]
    username = "deskguest"
    password = "deskguest"

```

If you are using the Cybersecurity Lab for testing these changes should be enough to get the project running.

Testing

Running Backend Tests

Note: Currently some backend tests are skipped, but the majority are enabled.

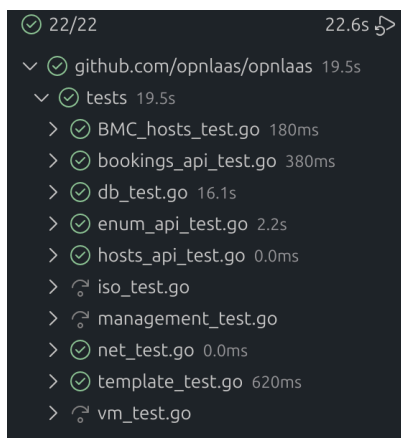
There are a few ways to run backend tests:

Locally

You will need go installed to run the tests locally. Tests can be run in one of two ways:

In VS-Code:

- Go to the testing menu on the (typically) left sidebar and hit the playbutton for the top test
- This will start all of the tests locally, you can view progress on the right and output in the test results tab in VS-Code



In a Unix Terminal:

- cd into the project directory and run `go test ./tests/`
- All tests will run locally, if you want logs / output you will have to run `go test` from within the `tests/` directory.

```
m_g@HP-Envy-x360-2-14z-fa000:~/Documents/UNH/2025-2026/CS-Work/518/opnlaas$ go test ./tests
ok      github.com/opnlaas/opnlaas/tests    20.366s
```

On Github

All of the backend tests are automatically run by a github actions agent on a PR change or any commit to main. There are a few minor differences between the two but that is due to github actions.

- The github actions agent workflows are configured in the `.github/workflows`

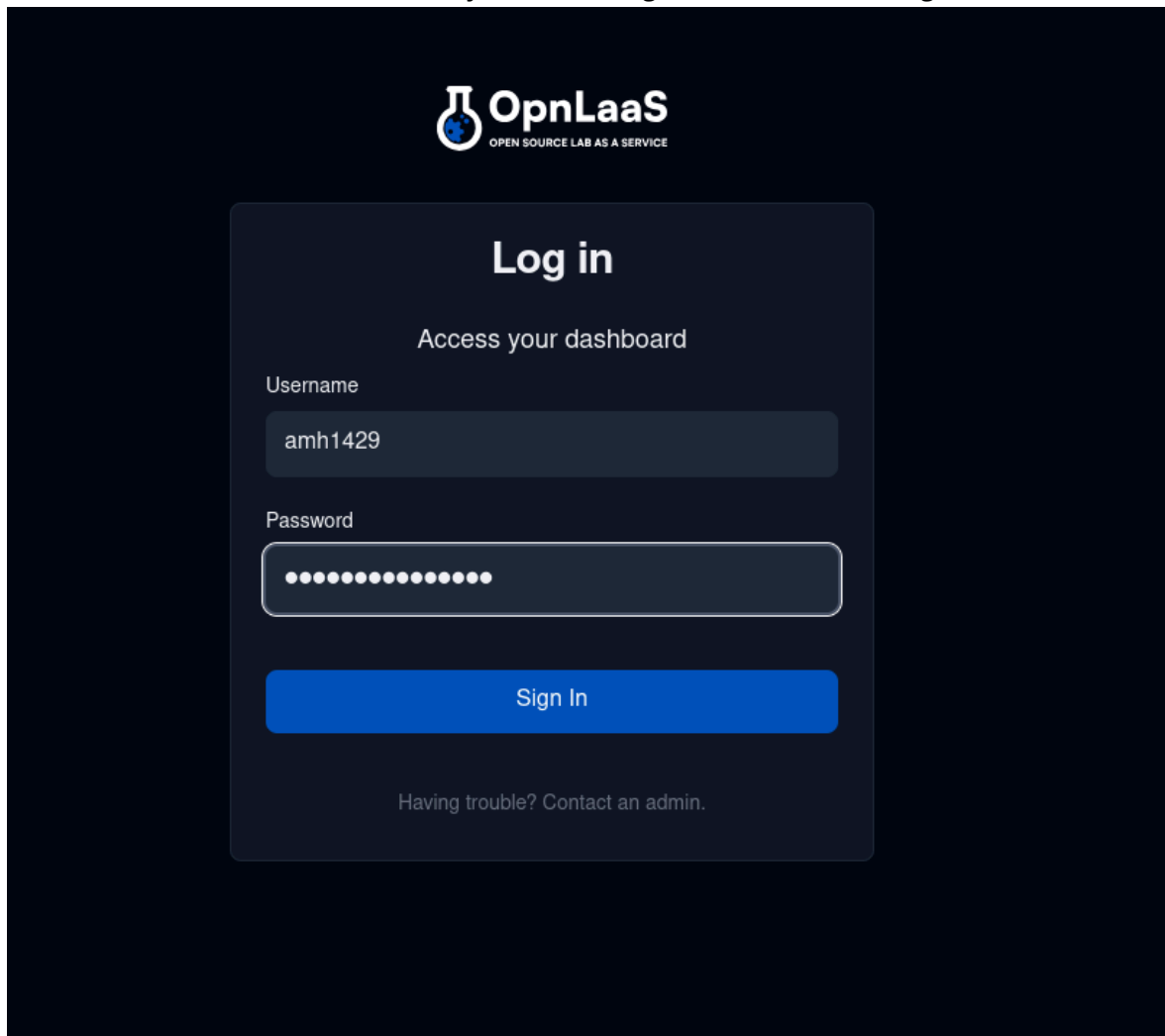
Running Frontend Tests

Due to the complexity of some of our frontend features we didn't implement automated testing for them. Below is documentation on how to use the 2 primary features we worked

on during this spint.

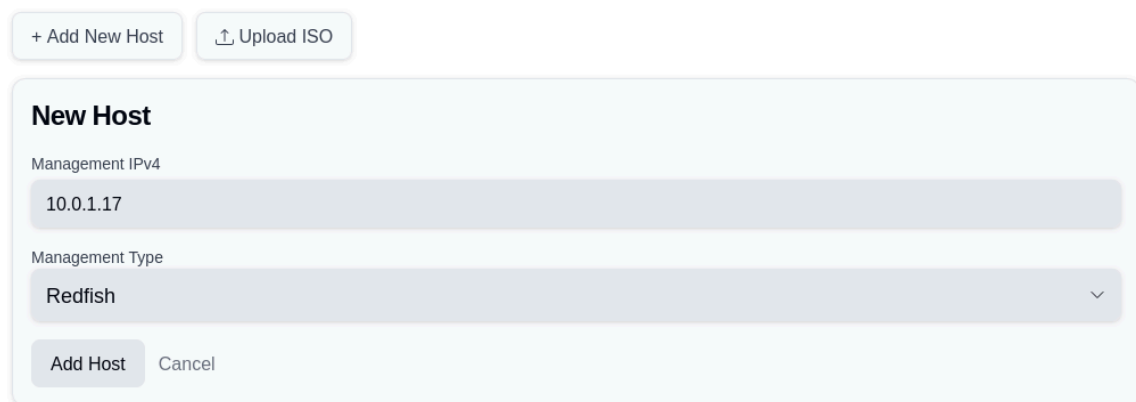
Feature 1: Resource Dashboard

To access the Admin Dashboard first you have to log in as an Admin at /login.



The image shows a login interface for OpnLaaS. At the top, the OpnLaaS logo is displayed with the tagline "OPEN SOURCE LAB AS A SERVICE". Below the logo, the text "Log in" is prominently shown, followed by "Access your dashboard". The login form consists of two input fields: "Username" with the value "amh1429" and "Password" with masked characters. A blue "Sign In" button is positioned below the password field. At the bottom of the form, there is a link that says "Having trouble? Contact an admin."

To add new host first you have to click the add new host button then enter an the IP address of the host and select whether it is managed by IPMI or Redfish. The IP address shown below is of one of our actual servers so that should work for testing purposes.



The image shows a "New Host" form. At the top, there are two buttons: "+ Add New Host" and "Upload ISO". Below these buttons, the form is titled "New Host". It contains two input fields: "Management IPv4" with the value "10.0.1.17" and "Management Type" with the value "Redfish". At the bottom of the form, there are two buttons: "Add Host" and "Cancel".

To view hosts go to /hosts

Hosts

PowerEdge M640 (VRTX) Other	● On As of 12/10/2025, 21:04	▼
PowerEdge R730 Other	● Off As of 12/10/2025, 21:04	▼
PowerEdge R730 Other	● Off As of 12/10/2025, 21:04	▼
PowerEdge R730 Other	● Off As of 12/10/2025, 21:04	▼
PowerEdge M640 (VRTX) Other	● On As of 12/10/2025, 21:04	▼

To power control hosts you click the three dot and then press select on option. **NOTE: You will need to be logged in as an administrator to see this menu.**

Hosts

PowerEdge M640 (VRTX) Other	● On As of 12/10/2025, 21:20	⋮ ▼
PowerEdge R730 Other	● Off	▼
PowerEdge R730 Other	● Off As of 12/10/2025, 21:20	⋮ ▼

Power On

Power Off

Graceful Shutdown

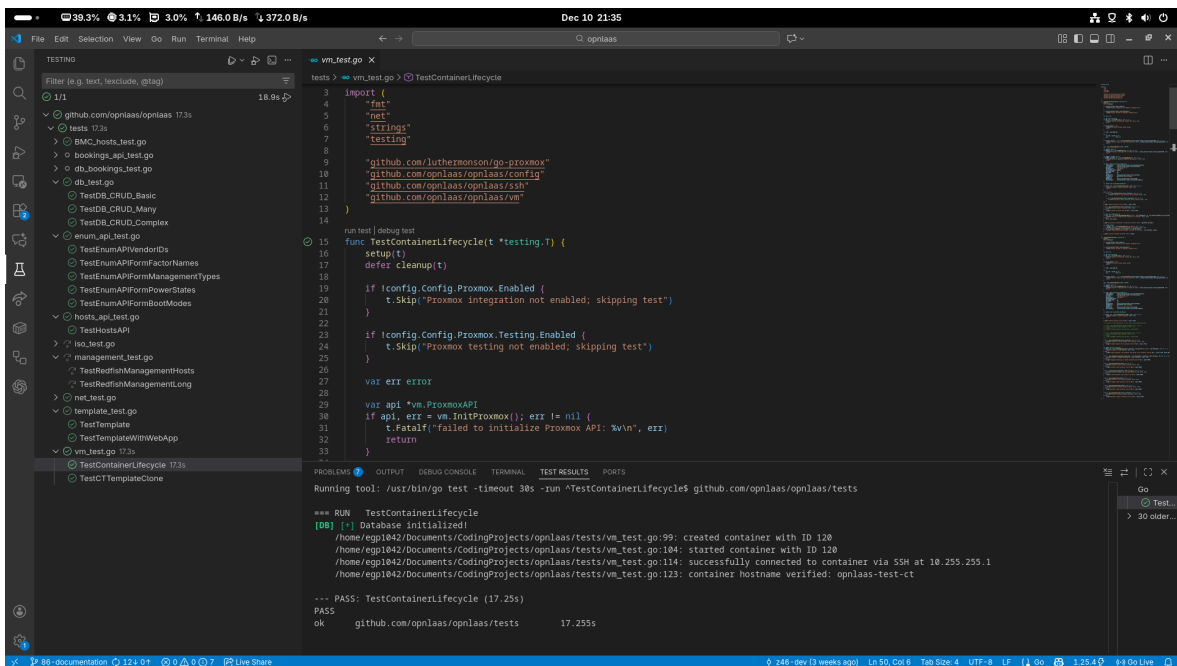
Graceful Restart

Force Restart

Unenroll Host

Feature 2: Virtualization Integration

Unlike the resource dashboard our virtualization integration with proxmox is a bit more complicated. As of now we haven't implemented this on the frontend which means you will need to test this functionality from the backend. In our testing folder we have a file called `vm_test.go` which includes testing for this. If you run this test you should see something like this.



Then when viewing the logs from proxmox (running at `proxmox.cyber.lab:8006`) you can see that containers are being created and deleted remotely:

